# Web System Technologies




## IT0049



MODULE 7

# Laravel Forms, Validation, and File Upload

# Objectives

*By the end of this session, you will be able to:*

- **Build Laravel form submission routes**
- **Use request validation and display validation feedback**
- **Redisplay old input after validation errors**
- **Update records through controller methods**
- **Validate, store, display, and replace uploaded files safely**

# What We'll Cover

## Subtopic 1

**Forms and Request Validation**

- GET and POST routes
- CSRF protection
- Errors and old input

## Subtopic 2

**CRUD and File Upload**

- Edit and update workflows
- Storage and file validation
- Displaying uploaded files safely

SUBTOPIC 1

# Forms and Request Validation

# Two Routes Per Form

A form workflow usually needs one route to show the form and another route to receive the submitted data.

GET displays the form page.

**POST sends the user's input to the controller.**
**The controller validates the input before storing or updating records.**

# Create and Store Routes

First generate the controller. The next slide registers the GET and POST routes before you verify them.



# Write the Form Routes

Write the routes in routes/web.php:

routes/web.php

```php
use App\Http\Controllers\TaskController;

Route::get('/tasks/create',
    [TaskController::class, 'create']
)->name('tasks.create');

Route::post('/tasks',
    [TaskController::class, 'store']
)->name('tasks.store');
```

# CSRF Token Required

Laravel protects web forms from cross-site request forgery.

- **Every POST, PUT, PATCH, or DELETE form in web routes should include @csrf.**
- **Without it, Laravel rejects the form submission.**

# Build a Create Form

The form posts to the named store route:

```
resources/views/tasks/create.blade.php

<form method="POST" action="{{ route('tasks.store') }}">
    @csrf

    <label>Title</label>
    <input name="title" value="{{ old('title') }}">

    <button type="submit">Save</button>
</form>
```

# The Controller Receives the Request

**Laravel can inject the current HTTP request into a controller method.**

TaskController.php

```php
use Illuminate\Http\Request;

public function store(Request $request)
{
    // Validate the submitted form data here.
}
```

# Validate Before Saving

- The validate() method checks the incoming request.
- If validation fails, Laravel automatically redirects back with errors and old input.

# Write the Validation Rules

Rules should match the table columns and the application requirement:

TaskController.php

```
$validated = $request->validate([
    'title' => ['required', 'string', 'max:100'],
]);
```

*If validation fails, Laravel redirects back with errors.*

# Display Validation Feedback

- Blade can show errors beside the field that failed validation.
- @error('title') checks whether the title field has an error.
- {{ $message }} prints Laravel's validation message.
- old('title') restores what the user typed after a failed submission.
- This makes the form easier to correct without starting over.

**A good form workflow protects the database, guides the user, and keeps the input visible when something needs correction.**

SUBTOPIC 2

# CRUD and File Upload

# Edit and Update Follow the Same Pattern

**The edit page loads an existing record.**

- The form shows the current values.
- The update route receives the edited values.
- The controller validates the values before saving changes.

# Define Edit, Update, and Upload Routes

routes/web.php

```php
Route::get('/tasks/{task}/edit',
    [TaskController::class, 'edit']
)->name('tasks.edit');

Route::patch('/tasks/{task}',
    [TaskController::class, 'update']
)->name('tasks.update');

Route::post('/tasks/{task}/upload',
    [TaskController::class, 'upload']
)->name('tasks.upload');
```

# Load the Existing Record

The edit method sends the selected record to the form view:

TaskController.php

```php
use App\Models\Task;

public function edit(Task $task)
{
    return view('tasks.edit', [
        'task' => $task,
    ]);
}
```

The URL value selects the record.

The controller sends the record to Blade.

The form shows the current values.

# CI vs. Laravel Form Workflow Map

## CodeIgniter pattern

- Show form from controller
- Validate submitted input
- Return errors to view
- Update model record

## Laravel pattern

- GET route + controller method
- **$request->validate([...])**
- @error and old() in Blade
- Eloquent update([...])

# Write the Update Method

Validate first, then save the changes:

TaskController.php

```php
public function update(Request $request, Task $task)
{
    $validated = $request->validate([
        'title' => ['required', 'string', 'max:100'],
    ]);

    $task->update($validated);

    return redirect()->route('tasks.index');
}
```

# Build the Edit Form

The edit form posts to the update route and keeps the current title visible.

tasks/edit.blade.php

```
<form method="POST" action="{{ route('tasks.update', $task) }}">
    @csrf
    @method('PATCH')

    <input name="title" value="{{ old('title', $task->title) }}">

    <button type="submit">Update</button>
</form>
```

# File Upload Form

tasks/edit.blade.php

```
<form method="POST"
      action="{{ route('tasks.upload', $task) }}"
      enctype="multipart/form-data">
    @csrf
    <label>Attachment</label>
    <input type="file" name="attachment">
    <button type="submit">Upload</button>
</form>
```

*The action points to the upload route; enctype makes the browser send the file bytes.*

# Validate Uploaded File

File validation should restrict type and size before storage:

TaskController.php

```php
$validated = $request->validate([
    'attachment' => [
        'nullable',
        'image',
        'mimes:jpg,jpeg,png',
        'max:2048',
    ],
]);
```

*max:2048 means 2048 KB, or about 2 MB.*

# Store the File on the Public Disk

Save only the file path. The task record already exists, so update() adds the attachment field.

TaskController.php

```
if ($request->hasFile('attachment')) {
    $path = $request->file('attachment')
        ->store('attachments', 'public');
    $task->update([
        'attachment' => $path,
    ]);
}
```

# File Upload Commands

```
php artisan storage:link
```

- Expose public storage files through /storage

```
$request->hasFile('attachment')
```

- Check that a file was submitted

```
store('attachments', 'public')
```

- Save the uploaded file on the public disk

```
Storage::disk('public')->delete($path)
```

- Remove old files when replacing them

# Display the Uploaded File

After storage:link, public disk files can be shown using the /storage path:

tasks/show.blade.php

```
@if ($task->attachment)
    <img
        src="{{ asset('storage/'.$task->attachment) }}"
        alt="Task attachment"
        width="120"
    >
@endif
```

# Replace an Existing Uploaded File

When users upload a replacement, delete the old file after the new file is accepted:

TaskController.php

```php
if ($request->hasFile('attachment')) {
    if ($task->attachment) {
        Storage::disk('public')
            ->delete($task->attachment);
    }

    $path = $request->file('attachment')
        ->store('attachments', 'public');
}
```

# Prepare Images for Display

After validation, resize or crop the uploaded image before storing the display path.

TaskController.php

```php
use Intervention\Image\Laravel\Facades\Image;

$name = uniqid('avatar_') . '.jpg';
$path = storage_path('app/public/attachments/' . $name);

Image::read($request->file('attachment'))
    ->cover(300, 300)
    ->save($path);
```

# Form Request Classes Keep Controllers Clean

For longer validation rules, move the rules into a Form Request class.

```
Terminal

php artisan make:request UpdateTaskRequest

# In the request class:
public function rules(): array
{
    return [
        'title' => ['required', 'string', 'max:100'],
    ];
}
```

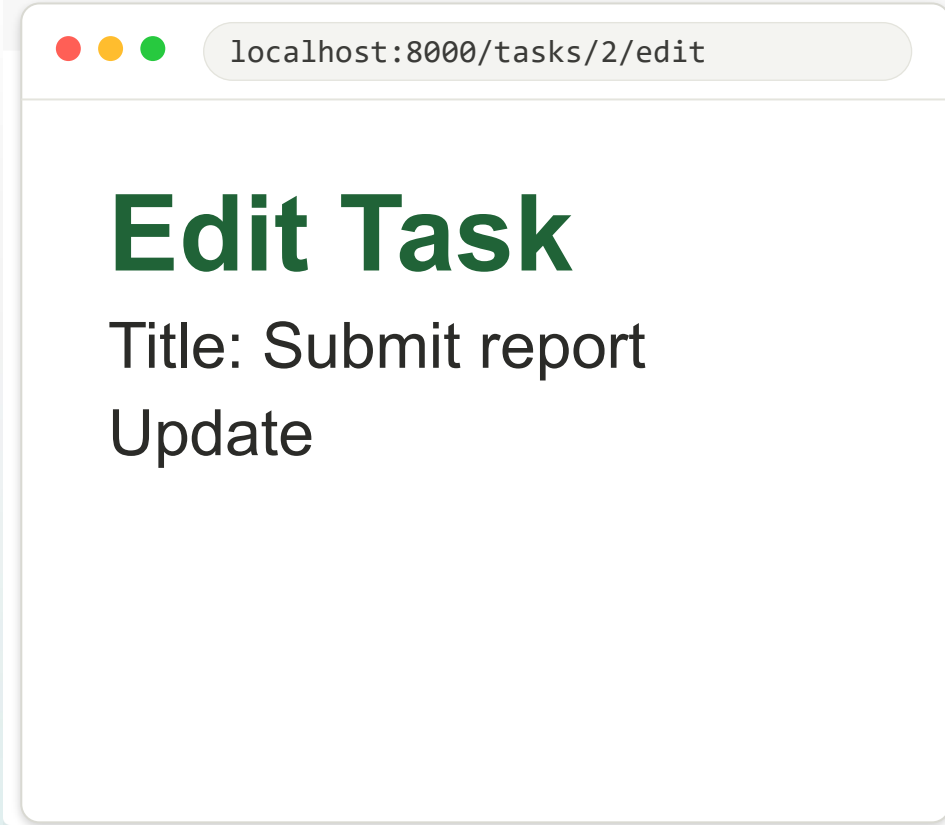
# Putting It Together: Edit Task

After validation, the controller updates the record and returns to the list:

TaskController.php

```
$task->update($validated);

return redirect()
      ->route('tasks.index');
```



# Putting It Together: Upload Flow

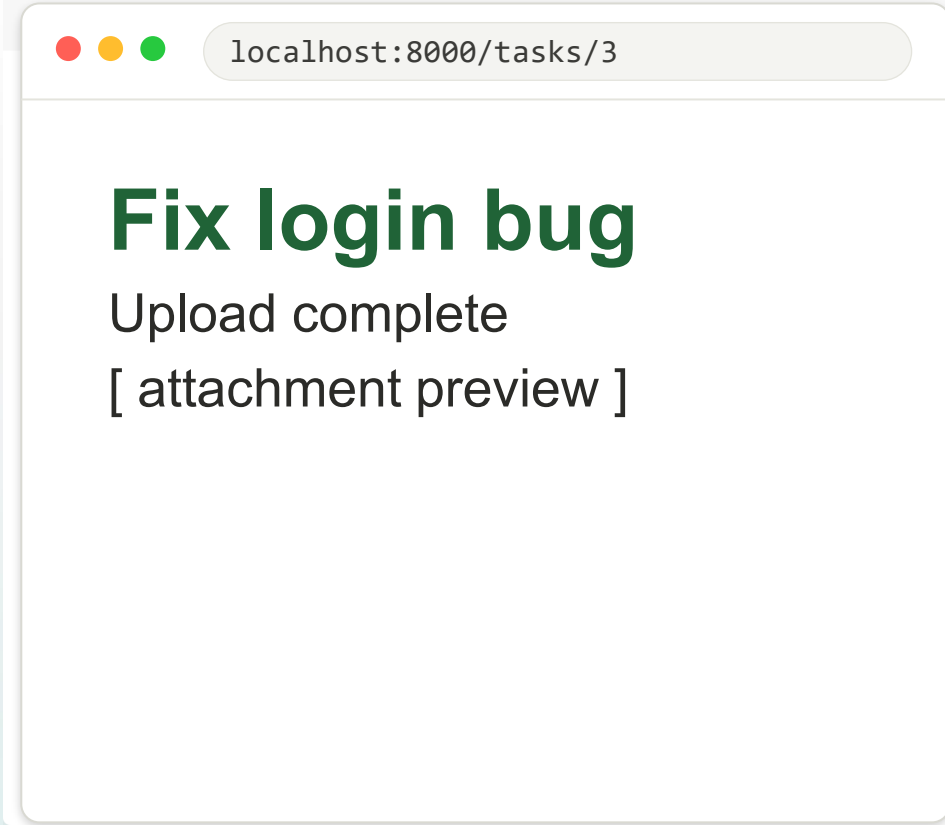
The final upload method stores the file, saves the path, and redirects.

TaskController.php

```php
$path = $request->file('attachment')
    ->store('attachments', 'public');

$task->update([
    'attachment' => $path,
]);

return redirect()->route('tasks.show', $task);
```



# Safe Upload Checklist

Use this order: validate, store, save the path, display safely, and clean up old files.

**Security habit**

```
$request->validate(['attachment' => [
    'nullable', 'image', 'max:2048'
]]);

// Do not trust the original filename.
// Prepare image size before display.
// Save only the generated path.
// Delete old files only after replacement succeeds.
```

# Common Mistakes & Key Takeaways

## Watch out for

- Missing @csrf
- Wrong form method or action
- Forgetting @method('PATCH')
- No enctype for file upload
- Saving files before validation

## Remember

- GET shows forms
- POST/PATCH submits data
- validate() redirects with errors
- old() keeps user input
- Storage manages uploaded files

# Q&A SESSION

Questions about Laravel forms, validation feedback, update workflows, or file uploads?

